



Practical Course - Operating Systems - seL4 & TRENTO

W4 – CAmkES

Copyright Notice

Copyright

- © 2021 HENSOLDT Cyber GmbH – All rights reserved.

Trademarks

- TRENTOS® is a trademark of HENSOLDT Cyber GmbH.
- seL4® is a trademark of LF Projects, LLC.

Why is using seL4 difficult?

- it's still a small eco system
 - most people are advanced users
 - projects happen in highly specific domains or academia
 - lack of industrialization, so normal developers can use it
- Example: understanding seL4 capabilities
 - the whole seL4 system is built on them
 - „a capability is a key that conveys specific rights to a particular object“
 - Object types (C-Spaces, C-Nodes, untyped memory, TCBs, ...)
 - Operations (Invoke, Copy, Mint, ...)

seL4 Capability Management

- „I spent 18 months on this problem“ (intern, 2013, about a lib for cap allocation)
- Involves lots of manual work for systems: do the initial setup, align with system changes and review everything
- It's a very technical task
 - basic patterns repeat over and over – with subtle differences
 - tends to end up in „copy, paste & patch“
 - magic patterns spread (partly related to poor documentation)
 - attic patterns don't die out
- There are helper libraries, but ...
 - created to provide compatibility with non-seL4 concepts, e.g. POSIX, Tasks, ...
 - lack of features when it comes to details, need multiple libs
 - implementation quality and maintenance varies

Why CAmkES?

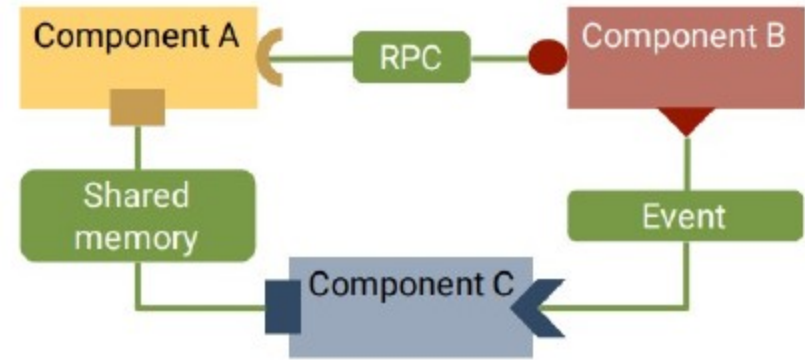
- Goal: build reliable and trustworthy systems
 - Microkernel-based systems promote the separation of functionality into separate services
 - Strong synergy between the approach of microkernel-based system design and CBSE (Component-based software engineering)
- Idea: modeling a system as interacting components fits the model of an operating system as a set of interacting services
 - Microkernel provides the required features (protection and communication)
 - Architecture for building componentised microkernel-based systems on top
- Result: layered component architecture for separation of concerns called CAmkES (Component **A**rchitecture for **m**icro**k**ernel-based **E**mbedded **S**ystems)

„a software development and runtime framework for quickly and reliably building microkernel-based multiserver (operating) systems“

What is CAmkES?

- Building Blocks

- a language to describe
 - component interfaces
 - components
 - whole component-based systems
- a tool that
 - processes the descriptions
 - combines programmer-provided component code with generated glue code
 - builds a complete, bootable system image
 - generates proofs
- full integration in the seL4 environment and build system
- ... sounds familiar?
 - UML, Model Driven Development, ...



CAmkES Requirements

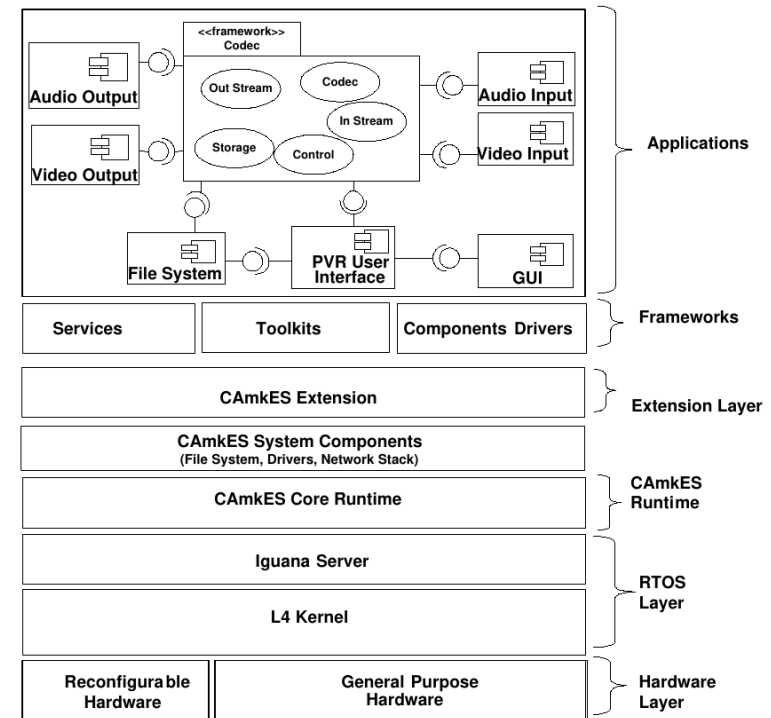
- System must be designed and built as a highly structured, componentised software architecture with
 - individual system services contained in separate components
 - well defined interfaces, explicitly specifying any interaction points
 - clearly detailing interaction between components
- Architecture provides a component model, standard interfaces and component definitions, component implementations, standard services and support for various architectural patterns
- Expectation
 - Architecture must make use of mechanics provided by the OS, e.g. IPC, memory management and protection (no re-implementation of services)
 - Mechanisms provided by the architecture must be efficient enough to be used by OS components (no significant performance penalties for the rest of the system)

CAmkES Advantages

- Keeps TCB small and manageable
- Fault containment by placing components in separate protection domains
- Exchangeable system architecture at runtime (loading/unloading of components)
 - hot-swapping of service implementations
 - dynamic update of OS services
 - detection and restarting of faulty services (including device drivers)
- Well-structured design with many small components supports safety, reliability and verification
 - Components are isolated
 - Interaction only using mechanisms provided by the microkernel
- But: many small components lead to more communication (performance impact by IPC)

CAMkES History

- **Hardware layer** (includes CPU, memory bus, ...)
- **RTOS layer**
 - Microkernel (OKL4)
 - Supervisory OS (L4/Iguana)
- **CAMkES core runtime**
- **Extension layer** (more advanced component features on top of the core runtime)
- **Frameworks** (provide further extension of functionality, components and services for specific application domains)
- Combination of user-defined components and underlying layers forms complete **application**

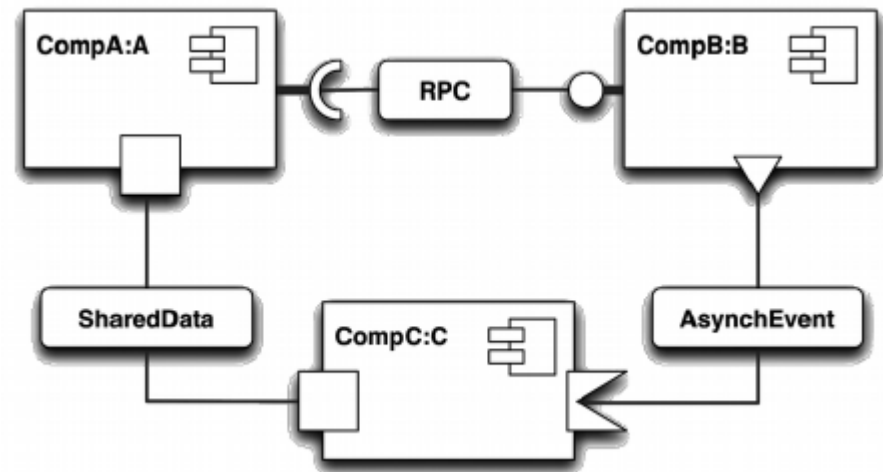


CAMkES Architecture

- CAMkES core runtime: execution environment providing the basic services required to deploy CAMkES components
- Supports only static components and component compositions
 - component instances are only created at system initialisation (i.e. boot) time
 - connections between components are established at design time (no creation or modification at runtime)
- Main features
 - Modularity: only features that are really needed for any particular target application (otherwise: extensions)
 - Simplicity: lightweight, focusing only on static components and their composition (minimising overhead)
 - Predictability: for static components, stubs and glue code are inserted into the components at compile time (allowing analysis of temporal behavior)

CAMkES Architecture

- CAMkES systems consist of components and connections between these components
 - each component has a collection of incoming and outgoing interfaces
 - connections must link an outgoing interface of one component with an incoming interface of another component
- Architectural elements
 - Components
 - Connectors & Connections
 - Compositions
 - Configuration
 - Assembly



CAmkES Components

- Services and applications as „black box“ with a well defined interface
- Component types
 - Active components: own thread of control (keyword **control**), run a kind of *main()* function
 - Passive components: get invoked from interface, e.g. RPC call, notification, interrupt
- Components are isolated from each other (unless explicitly modeled differently)
- Components can be instantiated multiple times
- Hardware peripherals are also modeled as component instances (keyword **hardware**)

CAMkES Connectors & Connections

- Support for three interface types
- Remote procedure calls (RPCs)
 - synchronous/blocking function invocation
 - keywords ***provides*** or ***uses***
- Events
 - supports a publish/subscribe event model
 - asynchronous notifications between components
 - suitable for interrupts
 - keywords ***emits*** or ***consumes***
- Dataports
 - bidirectional communication via shared memory
 - suitable for passing large data between components
 - true data sharing instead of copying (reduced performance overhead)

CAMkES Configuration

- Attributes (values represent the component's status or settings), capability properties (badges, priorities) or hardware settings (address, interrupt)
 - Attributes are accessible to the component at the code level at runtime
 - Values are not specified inside the component definition, but in a separate configuration specification when components are assembled
- Provide a way of adding extra information to a component or specialising the component in a particular context

CAMkES Compositions & Assembly

- Compositions
 - contain all components of the system and the connections that define their communication with each other
 - provide virtual interfaces as alias for interfaces of internal component interface
- Assembly: forms a complete system by
 - instantiating component types that have been defined and interconnecting these instances (composition)
 - specifying a system configuration

CAmkES Syntax

- Interface Definition Language (IDL)

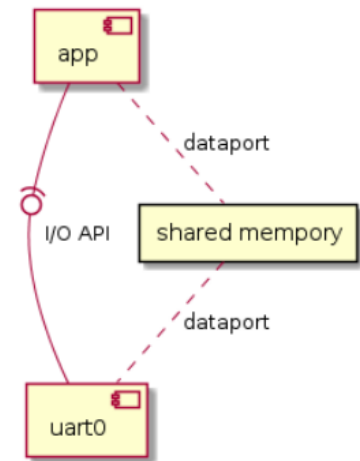
```
procedure interface_io {
    void write(size_t len);
    void read(size_t len);
};
```

- Architecture Description Language (ADL)

```
Component UART {
    provides interface_io io;
    dataport Buf          port;
}

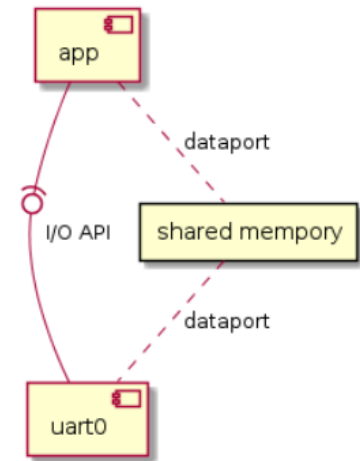
component APP {
    control;
    uses interface_io uart_io;
    dataport Buf          uart_port;
}

assembly {
    composition {
        component UART  uart0;
        component APP   app;
        connection seL4RPCall rpc(from app.uart_io, to uart0.io);
        connection seL4SharedData shareMem(from uart0.port, to app.uart_port);
    }
}
```



CAmkES Glue Code

- provides functions you can call to use the connectors
 - a function **uart_io_write()** can be called from the **app** to use the I/O RPC interface provided by **uart0**
- calls functions you have to implement
 - a function **run()** in the **app** that implements “the app”
 - a function **io_write()** in **uart0** that is called when the **app** calls **uart_io_write()**
- provides helper variables
 - A variable holding a pointer to the shared memory, it's **uart_port** in **app** and **port** in **uart0**



```

procedure interface_io {
    void write(size_t len);
    void read(size_t len);
};

Component UART {
    provides interface_io io;
    dataport Buf          port;
}

component APP {
    control;
    uses interface_io  uart_io;
    dataport Buf       uart_port;
}

assembly {
    composition {
        component UART  uart0;
        component APP   app;
        connection seL4RPCCall rpc(from app.uart_io, to uart0.io);
        connection seL4SharedData shareMem(from uart0.port,
                                           to app.uart_port);
    }
}
  
```

CAMkES mapping to seL4

- Built on top of the seL4 microkernel to build componentised systems with strong isolation guarantees
 - Ideal for high assurance environments due to the capability-based kernel
 - Provides well-defined boundaries between components by using the address space isolation features of seL4
- Abstracts the low-level kernel mechanisms of seL4 and provides communication primitives as well as support for decomposing a system into functional units
- Internally, CAMkES implements these abstractions using seL4's low-level kernel objects
 - Components comprise (at least) one thread, a CSpace and a VSpace
 - RPC connectors use endpoint objects and CAMkES generates glue code to marshal and unmarshal messages and sends them over IPC endpoints
 - A dataport connector is implemented using shared memory (shared frame objects present in the address spaces of two components, optionally restricting the direction of communication)
 - An event connector is implemented using seL4's notification mechanism

System Startup

1. A CAmkES system uses a code generator that parses the given ADL description and generates glue code
2. The glue code implements the interfaces as described in the ADL in terms of underlying platform mechanisms
3. The glue code is then compiled together with the user's component code to form a collection of userspace applications
4. These applications are configured by an initialiser task that runs as the first application after the kernel has booted

Under the hood - CapDL

- Traditionally: capability distributions are implicitly defined by the code that creates objects and transfers capabilities between subjects at runtime
- Problem: security analysis becomes a challenging task
- Solution: explicit representation of the system's capability distribution to allow for formal verification, easier debugging and better documentation
- CapDL as a language for explicitly describing the capability distribution of a seL4-based system
- CAMkES generates, in the CapDL language, a low-level specification of the system's initial (access control) configuration of kernel objects and capabilities
- CapDL specification is the input for the generic seL4 initialiser, which runs as first user task after boot to
 - instantiate and
 - initialise the system
- The first user task has access to all available memory and creates objects and capabilities according to the detailed CapDL description

Under the hood - CapDL

- **Capability Distribution Language**
 - formal language tightly coupled to seL4
- ADL + IDL is translated into a CapDL spec
 - precise representation of access rights in the system
- CapDL compiler
 - creates binary representation for system loader
 - creates C glue code
- Usually, you don't have to deal with CapDL directly

```
objects -- The object section starts here

DRIVER_ut = ut {
    DRIVER_tcb = tcb
    DRIVER_cspace = cnode (10 bits)
    DRIVER_code [5] = frame (4k)
    DRIVER_data [11] = frame (4k)
    DRIVER_vspace = pd
    DRIVER_pt = pt
}

CLIENT1_ut = ut {...}
CLIENT2_ut = ut {...}

DRIVER_aep = aep
IRQ_aep = aep
SHARED_frames [2] = frame (4k)

caps -- The capabilities section starts here

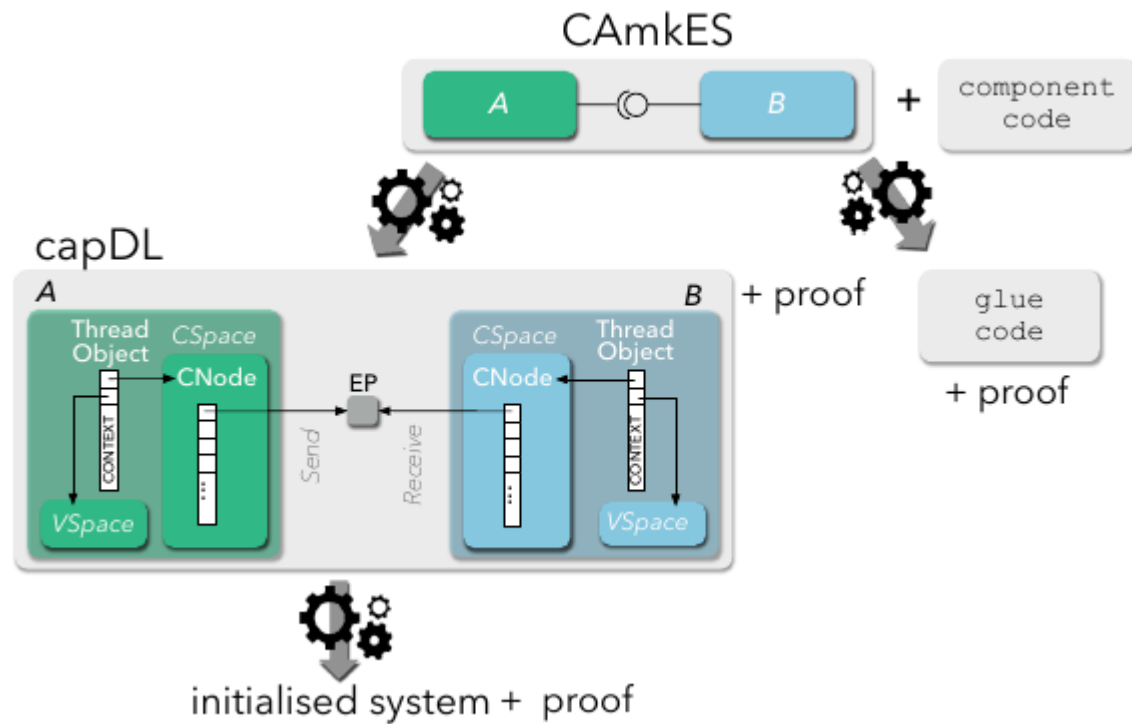
DRIVER_tcb = {
    cspace : DRIVER_cspace
    vspace : DRIVER_vspace
}

DRIVER_cspace = {
    1: DRIVER_aep (RW , badge : 0)
    2: IRQ_aep (R )
}

CLIENT1_cspace = { 1: DRIVER_aep (RW, badge : 1) }
CLIENT2_cspace = { 1: DRIVER_aep (RW, badge : 2) }
DRIVER_vspace = { 1: DRIVER_pt }

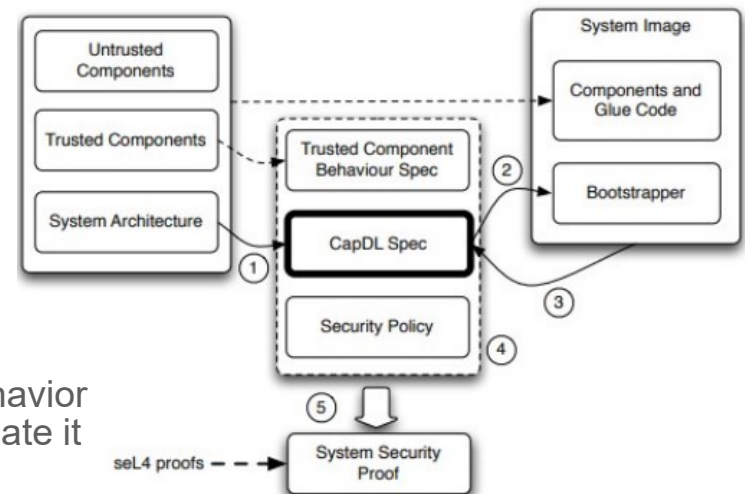
DRIVER_pt = {
    0x0 : SHARED_frames [] (RW)
    0x2000 : DRIVER_code [] (R)
    0x7000 : DRIVER_data [] (RW)
}
```

Under the hood - CapDL



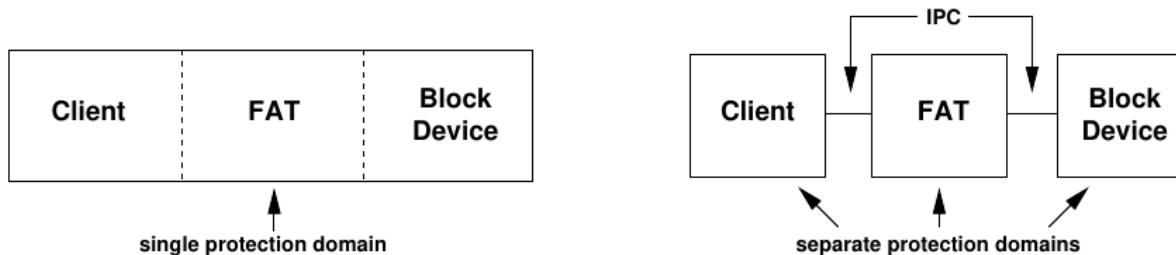
CapDL model

- CapDL model
 - has formal semantics in the theorem prover Isabelle/HOL
 - is connected to the seL4 specification
 - allows reasoning about the behavior of black-box components
- How to do formal verification
 - CAMkES generates
 - (1) the CapDL spec from ADL and IDL
 - (2) the system image and the CapDL loader
 - or (3) extract a CapDL spec from an existing system
 - (4) take a formalized security policy that specifies the behavior and use Isabelle to confirm the CapDL spec does not violate it
 - (5) take the existing seL4 proofs and use Isabelle to confirm the whole system implementation complies with the security policy



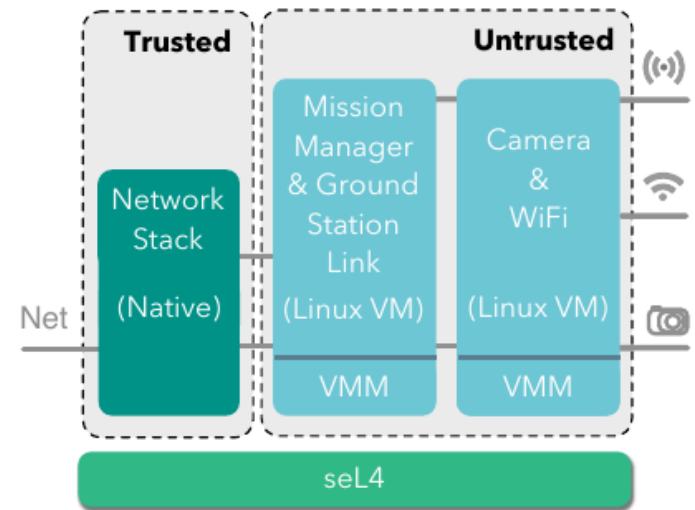
Example: Implementation possibilities for a library (e.g. FAT32)

1. Library is linked directly into a client program, which allows the client to directly access the filesystem without having to use IPC
 - library is linked into test program
 - all code runs in a single protection domain
2. Library is wrapped in CAMkES component code (FAT filesystem component)
 - client runs in a separate protection domain (IPC to invoke filesystem functions)
 - the underlying block device (which simply provides a block interface to a region of memory) also runs as a separate server in a different protection domain
 - components provide dataports used to transfer data during the read and write operations



Excursus: Virtualization on seL4

- Consists of CAMkES components that include a virtual machine monitor (VMM) and the guest OS (e.g. Linux)
- seL4 provides abstractions of the virtualisation hardware, while the VMM manages these for the VMs
- Isolation is ensured via separate CAMkES components per VM/VMM combination (allowing different assurance levels to coexist)
- Connectors model channels between VMs or from a VM to a native component
- seL4 constrains both the guest and the VMM (no need to trust VMM implementation for enforcing isolation)
- Failure of the VMM will lead to failure of the guest, but not to failure of the complete system
- VM may get access to hardware devices through para-virtualised drivers, pass-through drivers (e.g. via IOMMU) or both

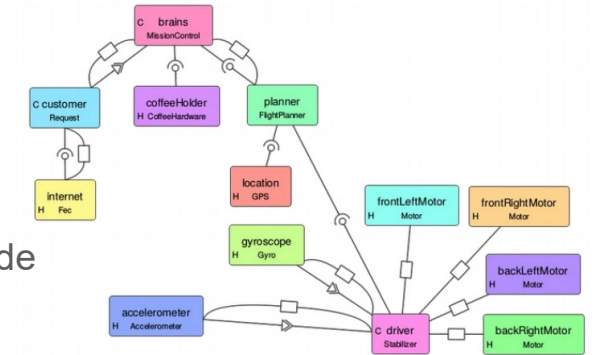


CAmkES languages

- CAmkES components can in principle be implemented in any programming language
- Nevertheless, security/safety-critical (trusted) components should be verified to ensure their trustworthiness
 - possible for C programs (as demonstrated by the verification of seL4)
 - but easier and more cost effective in higher-level languages
- CakeML: a ML dialect
 - Provides a certifying compiler as well as a runtime system that has been proved functionally correct
 - Used to build verified components of application code, as well as in the CapDL initializer
- Cogent: a simpler language, aimed at implementing systems code
 - No garbage collector, but type system ensures memory safety (like Rust)
 - Intentionally Turing-incomplete + additional library of (manually-verified) abstract data types
 - Cogent compiler produces C code + an Isabelle embedding of the program semantics and a proof certificate that the C code has the same semantics

CAMkES Goodies

- It works quite well
 - don't need to understand much of seL4
 - promise of proven functional correctness of the generated glue code
 - structured build workflow
 - visualCAMkES tool generates nice pictures
 - Rapid Prototyping ... assuming somebody already wrote nice connectors/components
- Device Tree Support
 - simply re-use the platform description language from Linux
 - automatic modeling of hardware components and their configuration
- DeviceDriver framework & Global Components
 - Standard components and connectors for common hardware and services
 - TimeServer (seL4 does not provide this in the API)
 - GPIO, Networking, Queues, Debugging/Benchmarking, ...



Advanced CAmkES

- Custom connectors
 - C glue code is created by a Python template processor
 - documentation could be better, learn by reverse-engineering the existing global components
 - ... also need to work on the proof
- Binary CAmkES
 - no strict requirement components must be built from C source code and use C glue code
 - a component is a black box, seL4 guarantees isolation – so any ELF files would work?

CAMkES drawbacks

- Supports only static systems
 - Can't install new components during runtime („reasoning about dynamic systems is hard“, PhD ongoing)
 - Components can't die or be restarted („verified components don't die“)
- CapDL loader sets up system using seL4 APIs
 - Simply uses the full seL4 functionality, because seL4 itself is verified for dynamic systems
 - Setting the capabilities during boot up takes time
 - System always ends up in exactly the same initial state (assuming HW does not change)
 - Could have a pre-created capability configuration snapshot and have the system read instantly, ... which requires changing seL4 and then updating the proofs
- Resource consumption
 - Optimize glue code, creates many threads and don't do much
 - Huge stack/heap default settings
 - Each component instance is a separate binary

CAMkES industrialization - TRENTOS

- Provide more ready-to-use components, especially drivers
- Improve connectors to hide more seL4 details (endpoint badges)
- Documentation and examples
- Integration in 3rd party tools (e.g. Eclipse)
- GUI based editor for system design
- Simulation, profiling and debugging
- Improve tool maturity and user-friendliness
- Apply proofs and verification

Sources

- p. 9, 24: Kuz et al. (2007): CAmkES: A Component Model for Secure Microkernel-based Embedded Systems
- p. 11: <https://ts.data61.csiro.au/projects/TS/camkes.pml>
- p. 21, 23: Kuz (2010): capDL: a language for describing capability-based systems
- p. 22, 25: Klein et al. (2018): Formally verified software in the real world
- p. 27: <https://docs.sel4.systems/projects/camkes/visual-camkes/>

HENSOLDT Cyber GmbH
Willy-Messerschmitt-Straße 3
82024 Taufkirchen
www.hensoldt-cyber.com

